MANUAL TÉCNICO: SISTEMA DE GESTIÓN DE NÓMINA

Grupo 02
Chinchilla Loaiza Ignacio
He Stefan
Rojas Esquivel Erika

Facultad de Tecnologías de Información y Comunicaciones
Universidad Latina de Costa Rica

Ingeniería en Sistemas Computacionales

BIS04 Programación II
C1-2026

Docente
Bryan Vega

San José, 22 de abril, 2026

Contenido

I. Descripción General del Sistema 3
II. Entorno de Desarrollo 3
III. Arquitectura por Capas 4
IV. Diagrama de Flujo del Sistema 4
V. Estructura del Código 6
5.1. Paquete vista 6
5.2. Paquete lógica 6
5.3. Paquete datos 7
VI. Persistencia de Datos - Archivos .txt 7
Ejemplo de lectura en un DAO 8
VII. Porcentajes Legales Aplicados 8
7.1. Cálculo del impuesto de renta 8
VIII. Manejo de Excepciones 9
IX. Dependencias Externas 9
X. Configuración SMTP 10

Contenido de Tablas y Figuras

Tabla 1. 3
Tabla 2. 4
Tabla 3. 6
Tabla 4. 6
Tabla 5. 7
Tabla 6. 7
Tabla 7. 8
Tabla 8. 9
Tabla 9. 9
Tabla 10. 10

Figura 1. 5

I. Descripción General del Sistema

El Sistema de Gestión de Nómina conómina cr es una aplicación de escritorio desarrollada en Java (JDK 21) con interfaz gráfica construida en Swing (JPanel). Su propósito es automatizar el ciclo completo de planilla de una empresa, aplicando la legislación laboral costarricense vigente para el cálculo de deducciones, aportes patronales e impuesto de renta por tramos.

El sistema opera exclusivamente con archivos de texto plano (.txt) como mecanismo de persistencia, sin uso de bases de datos relacionales. Cada entidad del sistema tiene su propio archivo separado, gestionado por una clase DAO dedicada que implementa la interfaz ManejadorArchivo.

El proyecto fue desarrollado con Apache NetBeans 21 IDE usando el sistema de construcción Java with Ant, siguiendo las convenciones de código Java (PascalCase para clases, camelCase para métodos), documentado con Javadoc en cada clase y método.

II. Entorno de Desarrollo

Tabla 1.
Componentes del Entorno de Desarrollo

Componente	Detalle
IDE	Apache NetBeans 21 (Java with Ant)
JDK	Java Development Kit 21
Lenguaje	Java 21
Interfaz gráfica	Java Swing (JPanel, JFrame, JTable, JComboBox, JSpinner)
Persistencia	Archivos de texto .txt con BufferedReader y BufferedWriter
Control de versiones	Git / GitHub — https://github.com/nachochin/Proyecto-Progra-2
Sistema operativo	Windows 10/11, macOS, Linux (multiplataforma)

Nota. Se muestra en la tabla los componentes que integran el Entorno de Desarrollo del Sistema. Elaboración propia, 2026.

III. Arquitectura por Capas

El sistema implementa una arquitectura de 6 capas, cada una con responsabilidades claramente delimitadas. Esta separación facilita la localización de errores, el mantenimiento del código y la escalabilidad futura del sistema.

Tabla 2.
Paquetes y clases del Sistema

Paquete	Clases principales	Responsabilidad
vista	VistaLogin, VistaPrincipal, VistaEmpleados, VistaNomina, VistaVacaciones, VistaConfiguracion	Interfaz gráfica Swing. Recibe entradas del usuario y delega a la capa lógica.
logica	CalculoNomina, Deducciones, CargasPatronales, GestionVacaciones, LogicaBase	Cálculos de nómina, deducciones, aportes patronales y gestión de vacaciones.
datos	EmpleadoDAO, NominaDAO, VacacionesDAO, UsuarioDAO, ConfigNominaDAO, ManejadorArchivo	Lectura y escritura de archivos .txt usando BufferedReader / BufferedWriter.
entidades	Empleado, Nomina, Vacaciones, Usuario, ConfigNomina, TramoRenta	POJOs con atributos, getters, setters y constructores. Sin lógica de negocio.
excepciones	AutenticacionFallidaException, ArchivoInvalidoException, CalculoNominaException	Excepciones personalizadas para autenticación, archivos y cálculos.
reportes	GeneradorPDFNomina, GeneradorPDFPatrono, EnvioCorreo, GeneradorReporte, VistaReportes	Generación de PDFs con iText y envío por correo con JavaMail.

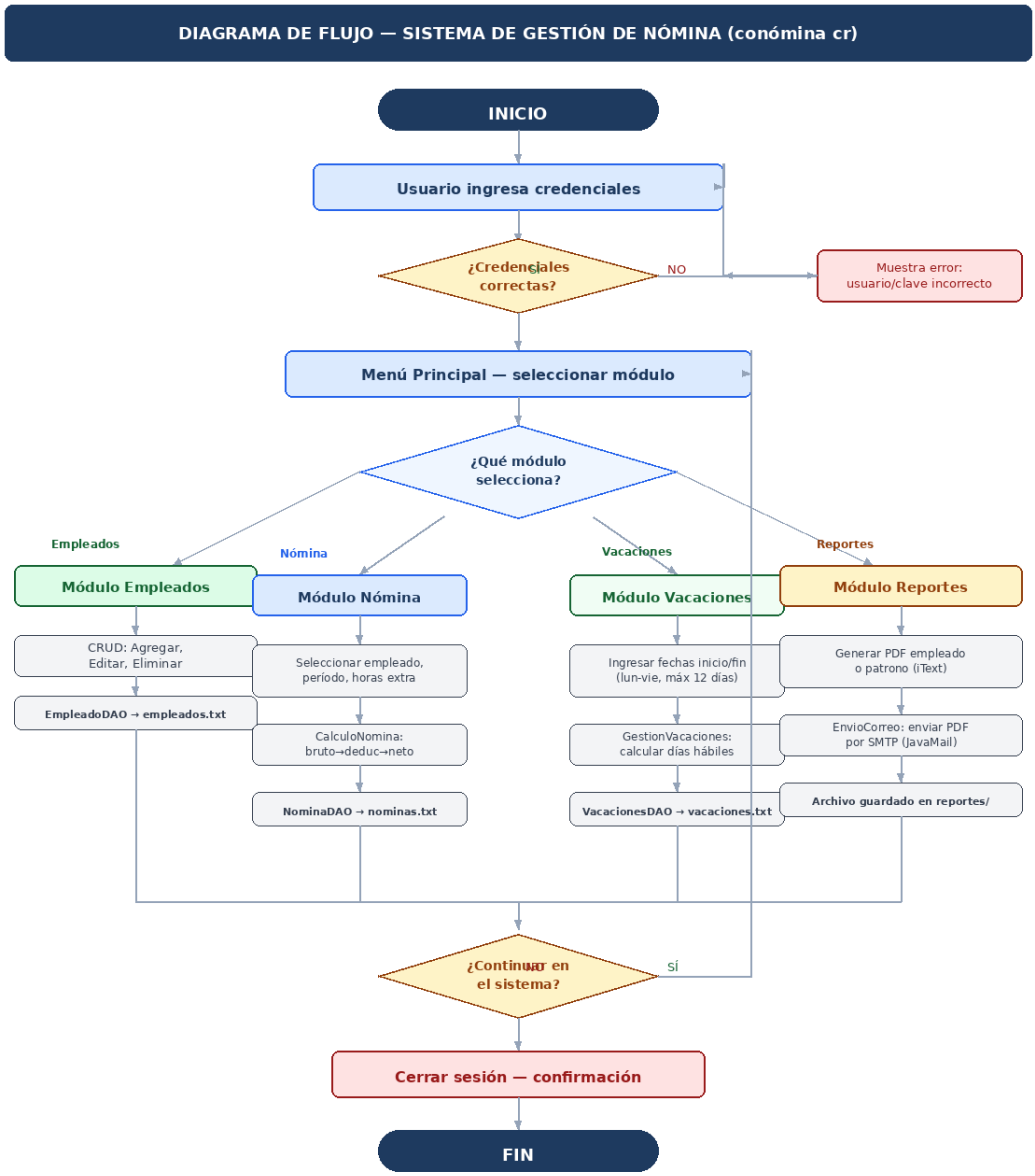
Nota. Se muestra en la tabla los paquetes y principales clases del Sistema. Elaboración propia, 2026.

La clase abstracta LogicaBase sirve como base para todas las clases de la capa lógica, inyectando la instancia de ConfigNomina y proveyendo el método utilitario redondear(). La interfaz ManejadorArchivo<T, ID> estandariza los métodos guardar(), buscarPorId(), listarTodos(), actualizar() y eliminar() en todos los DAOs.

IV. Diagrama de Flujo del Sistema

El siguiente diagrama representa el flujo completo de ejecución del sistema, desde el inicio de sesión hasta el cierre, incluyendo la navegación entre los cuatro módulos principales.

Figura 1.
Diagrama de Flujo del Sistema conómina cr



Nota. Se muestra el Diagrama de Flujo utilizado para el desarrollo del Sistema de gestión de Nómina. Elaboración propia, 2026.

V. Estructura del Código

5.1. Paquete vista

Tabla 3.

Estructura de código del Paquete vista

Clase	Responsabilidad
VistaLogin	Formulario de autenticación. Llama a UsuarioDAO.autenticar() y lanza VistaLogin o muestra el error.
VistaPrincipal	Ventana principal con barra lateral de navegación. Carga dinámicamente los módulos en el panel central.
VistaEmpleados	CRUD de empleados. Tabla central + formulario lateral. Inicializa el saldo de vacaciones al crear un empleado.
VistaNomina	Formulario de cálculo quincenal. Invoca CalculoNomina.procesarNomina() y muestra el desglose y el historial.
VistaVacaciones	Gestión de saldo vacacional. Calcula días hábiles (lun-vie), excluye fines de semana. Máximo 12 días/años no acumulables.
VistaConfiguracion	Edición de porcentajes de nómina y tabla de tramos de renta. Persiste cambios en config_nomina.txt y tramos_renta.txt.

Nota. Se muestra la estructura del paquete vista. Elaboración propia, 2026.

5.2. Paquete lógica

Tabla 4.

Estructura de código de Paquete lógica

Clase	Responsabilidad
LogicaBase (abstracta)	Clase base con ConfigNomina inyectada. Define el método abstracto calcular() y el utilitario redondear().
CalculoNomina	Orquesta el flujo completo: valida → calcula bruto → deducciones → aportes patronales → neto → persiste nómina.
Deducciones	Calcula CCSS, ROP, Banco Popular e impuesto de renta por tramos progresivos (base mensual proyectada / 2).
CargasPatronales	Calcula los 5 aportes patronales: CCSS, INA, Banco Popular, FCL y ROP. No se rebajan al empleado.
GestionVacaciones	Gestiona 12 días hábiles anuales no acumulables. Valida fechas (rechaza sábados/domingos). Método reiniciarSaldoAnual() para nuevo año.

Nota. Se muestra la estructura del paquete logica. Elaboración propia, 2026.

5.3. Paquete datos

Tabla 5.

Estructura de código de Paquete datos

Clase/Interfaz	Responsabilidad
ManejadorArchivo<T,ID>	Interfaz genérica que define los 5 métodos CRUD: guardar, buscarPorId, listarTodos, actualizar, eliminar.
EmpleadoDAO	CRUD sobre empleados.txt. Parsea LocalDate y Double. Lanza ArchivoInvalidoException si el formato es inválido.
NominaDAO	Clave compuesta: idEmpleado-anio-mes-quincena. Incluye listarPorEmpleado() para el historial.
VacacionesDAO	Persiste solo diasAcumulados y diasConsumidos. Los días restantes se calculan en tiempo real.
UsuarioDAO	Incluye autenticar() que compara la contraseña ingresada directamente con el valor almacenado.
ConfigNominaDAO	Maneja dos archivos: config_nomina.txt (formato clave=valor) y tramos_renta.txt (formato pipe).

Nota. Se muestra la estructura del paquete datos. Elaboración propia, 2026.

VI. Persistencia de Datos - Archivos .txt

Todos los archivos de datos se ubican en la carpeta data/ dentro del directorio raíz del proyecto. El separador de campos es el pipe (|). Los archivos se leen y escriben con BufferedReader y BufferedWriter respectivamente, usando el conjunto de caracteres UTF-8.

Tabla 6.

Persistencia de archivos .txt

Archivo	Formato de línea	Descripción
empleados.txt	id|nombre|correo|salarioBase|tipoContrato|fechaIngreso	Registro de empleados activos.
nominas.txt	idEmp|anio|mes|quincena|bruto|ccss|rop|bp|renta|otras|totalDed|patronal|neto|horasExtra	Historial de nóminas procesadas.
vacaciones.txt	idEmpleado|diasAcumulados|diasConsumidos	Saldo vacacional por empleado.
usuarios.txt	nombreUsuario|password|rol	Credenciales de acceso al sistema.
config_nomina.txt	clave=valor (ej: ccss_trabajador=0.0917)	Porcentajes configurables de nómina.
tramos_renta.txt	limiteInferior|limiteSuperior|porcentaje	Tabla de tramos del impuesto sobre la renta.

Nota. Se muestra el tipo de archivo y formato en la persistencia de datos. Elaboración propia, 2026.

Ejemplo de lectura en un DAO

```
try (BufferedReader br = new BufferedReader(new FileReader(RUTA_ARCHIVO))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        if (linea.trim().isEmpty()) continue;
        String[] campos = linea.split("\\|");
        if (campos.length != CAMPOS_ESPERADOS)
            throw new ArchivoInvalidoException("Formato inválido: " + linea);
        // construir objeto...
    }
}
```

VII. Porcentajes Legales Aplicados

El sistema aplica los porcentajes de cargas sociales establecidos por la legislación costarricense vigente. Todos los valores son configurables desde el módulo Configuración y se persisten en config_nomina.txt, por lo que no requieren modificación del código fuente cuando cambien las tasas.

Tabla 7.
Porcentajes legales aplicados al Sistema

Concepto	% Trabajador	% Patronal	Observación
CCSS (salud, IVM)	9.17%	26.33%	Se rebaja al empleado.
ROP (Pensión complementaria)	1.00%	3.25%	Se rebaja al empleado.
Banco Popular	1.00%	0.50%	Se rebaja al empleado.
INA	—	1.50%	Solo patronal.
FCL (Fondo Capitalización Laboral)	—	3.00%	Solo patronal.
Total cargas trabajador	**11.17%**	—	
Total aportes patronales	—	**34.58%**	

Nota. Se muestra el concepto y porcentaje aplicado a la nómina. Elaboración propia, 2026.

7.1. Cálculo del impuesto de renta

El impuesto se calcula por tramos progresivos sobre la base imponible mensual. Para nómina quincenal se proyecta el salario a mensual (× 2), se descuentan las cargas sociales para obtener la base imponible, se aplican los tramos y el resultado se divide entre 2.

Ejemplo del cálculo

```
double salarioMensual = salarioBruto * 2;
double baseImponible  = salarioMensual * (1 - config.getTotalCargasTrabajador());
double impuesto = 0;
for (TramoRenta tramo : config.getTramosRenta()) {
    if (baseImponible <= tramo.getLimiteInferior()) break;
    double enTramo = Math.min(baseImponible, tramo.getLimiteSuperior())
                     - tramo.getLimiteInferior();
    impuesto += enTramo * tramo.getPorcentaje();
}
return redondear(impuesto / 2); // monto quincenal
```

VIII. Manejo de Excepciones

El sistema utiliza tres excepciones personalizadas que extienden Exception, siguiendo el patrón checked exceptions de Java. Todas incluyen un constructor con mensaje y otro con mensaje + causa raíz (Throwable).

Tabla 8.
Excepciones personalizadas del Sistema

Excepción	¿Cuándo se lanza?	Clase que la lanza
AutenticacionFallidaException	Usuario no existe o contraseña incorrecta.	UsuarioDAO.autenticar()
ArchivoInvalidoException	Error al leer/escribir un archivo .txt o formato de línea inválido.	Todos los DAOs
CalculoNominaException	Salario cero, quincena inválida, período ya procesado o días de vacaciones insuficientes.	CalculoNomina, GestionVacaciones

Nota. Se muestra cantidad y tipo de excepciones personalizadas del Sistema. Elaboración propia, 2026.

IX. Dependencias Externas

Los archivos JAR se encuentran en la carpeta lib/ del proyecto y están referenciados en el archivo nbproject/project.properties de NetBeans. No requieren instalación adicional.

Tabla 9.
Dependencias externas del Sistema

Librería	Versión	Archivo JAR	Uso en el sistema
iTextPDF	5.5.13	itextpdf-5.5.13.5.jar	Generación de reportes en formato PDF.
JavaMail	1.6.x	mail.jar	Envío de correos electrónicos por SMTP.
Activation	1.1.x	activation.jar	Requerido por JavaMail para adjuntos.

Nota. Se muestra las dependencias externas utilizadas por el Sistema. Elaboración propia, 2026.

X. Configuración SMTP

El módulo de envío de correos usa el servidor institucional del curso. Las credenciales están definidas como constantes en la clase EnvioCorreo del paquete reportes.

Tabla 10.
Parámetros de configuración SMTP

Parámetro	Valor
Servidor SMTP	securemail.comredcr.com
Puerto	465 (SSL)
Protocolo SSL	TLSv1.2
Correo remitente	curso_progra2@comredcr.com
Propiedad mail.smtp.auth	true
Propiedad mail.smtp.ssl.enable	true

Nota. Se muestra los parámetros del SMTP utilizados por el Sistema. Elaboración propia, 2026.

Los correos se envían como mensajes multipart con el cuerpo en texto plano y el PDF adjunto. El sistema genera primero el archivo PDF en la carpeta reportes/ y lo adjunta al mensaje antes de enviarlo.